

LocalBuka Engineering Intern

Technical Assessment & Full-Stack Case Study

A practical evaluation package tailored to LocalBuka's core ecosystem

Target Role:	Software Engineering Intern (React, React Native, NestJS)
Time Limit:	5 Days (Take-Home)
Evaluation Focus:	Functional Completeness, TypeScript rigor, API Design, Mobile GPS Integration, Clean Git history
Submission Channel:	Send your GitHub repository link and walkthrough video to hellohr@localbuka.com

1. The Product Context

LocalBuka is a Nigerian restaurant discovery platform connecting hungry users with top chefs, neighborhood bukas, and premium dining spots. To test your ability to work on our production code, you will implement a lightweight prototype comprising parts of three core features:

- **Restaurant Discovery Feed:** Users can view nearby bukas, search by name, and filter by cuisine type, price, or rating.
- **Points & Rewards:** A gamified user engagement loop where users earn points for checks-ins or reviews, and redeem them for monetary vouchers.
- **GPS check-in:** Mobile users can verify physical presence at a restaurant within 100 meters to claim daily bonus points.

2. Backend API Requirements (NestJS / Express)

Your server must expose a set of clean, validated REST endpoints with consistent JSON payloads:

- **GET /restaurants:** List and search restaurants. If query parameters *latitude* and *longitude* are provided, calculate the distance in kilometers using the Haversine formula and return the list sorted from closest to furthest. Support filtering by *cuisine*, *priceRange* (1 to 4), and *minRating*.
- **GET /restaurants/:id:** Fetch detailed info on a single restaurant, including a child array of reviews.
- **POST /restaurants/:id/reviews:** Add a user review. Input must be validated (rating must be an integer between 1 and 5). *Crucial Constraint:* A user cannot review the same restaurant more than once; handle this conflict and return a descriptive error.
- **POST /points/earn:** Awards points to a user. Actions supported are *check-in* (50 pts), *review* (20 pts), and *referral* (100 pts). Ensure check-ins are restricted to once per calendar day per restaurant.

- **POST /points/redeem:** Redeems user points. Must validate that the user has a sufficient balance and that redemptions are performed in multiples of 50 points (e.g., 50 pts = N500 off).
- **GET /points/balance/:userId:** Returns the current aggregate points balance and a chronological transaction ledger of points earned and redeemed.

3. Frontend Web Requirements (React)

Build a responsive web application that interacts with your backend API. It should include:

- **Discovery Dashboard:** A list or grid of restaurant cards with search and filter controls. Implement skeletons/spinners for loading states.
- **Optimistic Favorites Toggle:** A button on the restaurant card to save/favorite it. The UI state must update immediately before the network request completes. If the request fails, the state must roll back gracefully and alert the user.
- **Voucher & Rewards Dashboard:** Displays user points and lists the ledger. Include a working voucher redemption section.

4. Mobile Screen Requirements (React Native) [Optional but Preferred]

Develop a simple React Native app screen using Expo or React Native CLI representing the **Restaurant Detail & Check-In** flow:

- Render the restaurant name, cuisine, rating, and a '**Check-In to Earn Points**' button.
- When tapped, request device location permissions. If granted, fetch the current GPS coordinates of the device.
- Compare the coordinates with the restaurant's location. If the user is within 100 meters, post to the backend `/points/earn`` check-in endpoint and display a success status animation. Otherwise, prompt the user that they are too far away.

5. Technical Assessment Questions

To test your understanding of core technologies, complete the following practical coding tasks and include the answers in your **SOLUTION.md**:

Q1. TypeScript & API Response Validation

Often raw JSON data returned from upstream or legacy endpoints contains inconsistent snake_case keys or string-formatted floats instead of numbers. Define a strict TypeScript type or interface representing the ideal **Restaurant** structure. Write a type-safe parser function `parseRestaurant(raw: unknown): Restaurant` that parses, casts, and validates raw payloads, throwing clear errors on missing fields or incorrect types. How would you integrate this with NestJS's global ValidationPipe using *class-validator* and *class-transformer*?

Q2. Concurrent Promises & Graceful Timeouts

To maximize search availability, we want to query our local database API and a fallback Google Places API simultaneously. Write a clean JavaScript/TypeScript utility function `fetchNearbyRestaurants(coords)` using Promise APIs that starts both requests concurrently. If the local database responds first, use it. If the local database fails, fallback to Google Places. Enforce a strict global timeout of 5000ms. If both fail or the timeout triggers, return a safe empty list and log the error context.

Q3. Race Conditions in Points Redemption

If a user rapidly taps the 'Redeem Voucher' button, there is a risk of a race condition where multiple requests arrive at the backend before the database can update the points ledger, potentially leading to double-redemption (spending more points than available). Describe two distinct approaches (one database-level, one API/service-level) to safely prevent race conditions in points balance checks.

Q4. React Performance & Component Rendering

In our search results page, rendering 200+ restaurant cards can cause severe UI lag on lower-end mobile devices during scrolling or filtering. Detail 3 specific optimization strategies you would apply to the React component tree (e.g. virtualization, memoization) to keep rendering smooth. How would you measure the performance improvements using browser DevTools?

6. Grading & Evaluation Rubric

Candidates will be assessed out of 100 points, broken down as follows:

Category	Weight	Core Grading Criteria
Backend API (NestJS)	40 Points	REST standards, schema design, index usage, input validation, daily duplicate check constraints.
Frontend App (React)	30 Points	State management choice, design aesthetics, responsive design, optimistic UI states with error recovery.

Mobile App (React Native)	15 Points	Permission Handling, GPS coordinates math, check-in validation, deep linking setup.
Engineering Quality	15 Points	TypeScript strictness (avoiding 'any'), clean Git commits, README accuracy, SOLUTION.md writeup.